

What CI covers — and why nothing says “this change hits NDF2W”

Business rules cannot be unit-tested properly. Neither the tests nor CI can tell you whether a change affects NDF4W underwriting or NDF2W. Nothing is checked until the service is already serving traffic. “Still running” is treated as “healthy”. And we look at what a nightly end-to-end run really proves.

2,286 TEST FILES ACROSS 4 REPOSITORIES. 4 OF THEM RUN A CAMUNDA PROCESS.	248,685 NDF2W APPLICATIONS A “NEW GENERATION” EDIT CAN REACH	38,198 WARNINGS A WEEK FROM THE ENGINE READING ITS OWN MODELS, IN PRODUCTION	6 of 9 TEST LAYERS MISSING — INCLUDING BOTH OF THE ONES THAT RUN IN MILLISECONDS
--	--	--	--

Seven claims, checked against the build and against production

Business rules cannot be unit-tested properly	● TRUE	And for a <i>sharper</i> reason than in LORA. 776 of 1,457 files are plain Mockito tests, and Mockito can reach Java. But Bravo's routing logic is not Java. It is <code>conditionExpression</code> strings inside 53 BPMN files, a six-column database selector, and a per-application on/off matrix in <code>jsonb</code> . All three are data, and no unit test runs any of them.
CI cannot tell an NDF4W underwriting change from an NDF2W one	● TRUE, AND WORSE	The product is identified in five different places at once . And <code>ndf2w.bpmn</code> calls into the <i>unified</i> underwriting sub-process, so a change to unified underwriting reaches NDF2W — 248,685 applications over 90 days — with nothing in the build saying so.
Nothing is checked until the service is serving traffic	● CONFIRMED	All 53 BPMN and 3 DMN files are deployed and read at Spring Boot startup. In 7 days the engine produced 38,198 ENGINE-09004 warnings in production , naming real modelling mistakes in real files. There is no check before that point.
“Still running” is treated as “healthy”	● CONFIRMED, IN BRAVO IT IS “LORA 5xx”	Here is in “LORA 5xx” that gets treated as healthy. Every success-rate monitor filters on <code>http.status_code:5*</code> . A flood of 748,686 404s a week, across 69% of surveyor sessions, counts as a healthy service by that definition.
A nightly end-to-end run proves the contract	● IT EXISTS — AND IT CAN BE CORRUPTED	Corrupted 2026-09-11. Bravo has a 595-file Cypress and Cucumber suite in a sibling repository, plus four CI workflows, one of them scheduled weekly. But every step ends with <code> true</code> , so the run can never go red . It covers 10 of the 595 files . And the repository has been frozen since 2023-11-21 . LORA's nightly run is mostly red, which is bad. Bravo's is permanently green by design, which is worse . Even if it ran: 57% of the Then steps in its 465 UI feature files only click and type . Its 130 API files do assert, on 87% of their steps, so the problem is confined to the UI half.
The 2,286-file suite is a safety net for the workflow	● NO	Four test files deploy and run a Camunda process, one of them against a production BPMN. Nothing tests what happens when retries run out, when an incident is raised, when an escalation crosses a <code>callActivity</code> , or when a loan is forcibly terminated.
There is a coverage gate	● THERE IS NO GATE	<code>pom.xml</code> sets JaCoCo to <code>report</code> and never to <code>check</code> , so there is no threshold. The <code>makefile</code> passes <code>-Dspring-boot.run.profiles</code> , which surefire ignores — so the tests are not running under the profile the build says they are .

Bravo’s tests cover the Java they can reach. They do not touch the XML, the config tables or the engine — which is where a loan’s behaviour is actually decided.

So the build can go green for a change that alters the routing of 94% of production volume.

ONE STRUCTURAL DIFFERENCE FAVOURS BRAVO

Bravo has 2,286 test files across four repositories, against LORA’s 608 across 13. LORA has **four repositories with no tests at all**, and lora-process-sdk has no .github/workflows/ directory whatsoever. Every Bravo repository that is not frozen runs its tests on every pull request. Bravo’s problem is that it covers the wrong things; LORA’s is that some repositories are not covered at all. Corrected 2026-09-10 — this panel used to say “Bravo has one repository, one build”, which was false. There are four, and the corrected totals widen the gap in Bravo’s favour.

THE GIVEAWAY

camunda-bpm-assert , camunda-bpm-process-test-coverage and camunda-bpm-mockito are all listed in pom.xml , and all three are effectively unused. **Somebody knew this was the right way to test a BPMN system, added the dependencies, and the practice never caught on** — the same shape as LORA’s lora-super-test , which has 150 scenario tests and nothing in CI to run them.

Sections 1 to 6 diagnose the problem. §7 is the proposed structure — nine layers ordered by cost, L0 to L8, a regression suite T1–T12, a failure suite N0–N12, and a rule that says which layer should have caught what.

2,286 test files across four repositories. Four of them run a process.

WHERE	COUNT
Test files in <code>ms-bpm</code> , under <code>src/test/java</code>	1,457
Test files in the three operator consoles – not counted until 2026-09-10	829
Plain Mockito unit tests	776
Tests that deploy and run a Camunda process	4
H2 “parse tests” that check BPMN structure	a handful
Full walks from application start to go-live	0
Tests of retries running out, incidents, escalation across a <code>callActivity</code> , forced termination	0
Coverage thresholds anywhere in the estate	none
Bravo LOS total	2,286
One of the four Camunda-running tests uses a production BPMN, <code>unified-anti-fraud-engine.bpmn</code> . JaCoCo is set up to produce a <code>report</code> only – there is no <code>check</code> goal and no minimum.	

0.27%

OF TEST FILES EXERCISE THE WORKFLOW ENGINE – AND THE ENGINE IS WHAT DECIDES WHAT HAPPENS TO A LOAN

The suite is large, well looked after and conventional. **Nothing in it is bad.** The problem is what it points at.

WHAT THAT LEAVES UNCOVERED

1,453 of the 1,457 `ms-bpm` files test the Java around the process, and none of the 829 console files reaches the process at all. The four that do start a process do not cover the bridge from the older workflows to the newer ones, the retry policies, or the decisions to degrade — and nothing else covers them either.

The layer this pack had not counted — and it does block merges

Team Bravo pointed out that Bravo LOS includes three operator-console repositories and the LN Jira project. **We checked, and they are right.** These are the same three applications already measured in browser monitoring at 2.37 million errors a week — visible in the runtime analysis, missing from the code analysis.

REPOSITORY	LINES OF CODE	TEST FILES	COMMITTS, 12 MO
bravo-surveyor-console v2.83.12	306,670	286	1,636
bravo-underwriting-console v1.63.13	249,366	441	1,493
bravo-operation-console v2.80.18	207,825	102	1,205
Total	763,861	829	4,334
All three were released within three days of this measurement. Across Bravo’s four LOS repositories, 25 different people committed in the last 12 months — not 7.			

A FAILING UNIT TEST BLOCKS THE MERGE

pr-pipeline runs lint , then unit-test , then compile_code , and compile_code declares needs: [unit-test] . There is no || true and no continue-on-error in any of the 28 console workflows. This is the strongest piece of testing practice in the Bravo estate — and the exact opposite of the OPERATION_PLATFORM.yml scheduled job, where every step ends in || true . Both patterns live in the same estate.

BUT COVERAGE IS MEASURED, NOT ENFORCED

The thresholds are set to 0% for surveyor and operation, and 1% for underwriting. Coverage is computed, written to lcov.info and sent to SonarQube — and no number can fail the build. jest-coverage-thresholds-bumper is installed and never called: the same shape as camunda-bpm-mockito . Sonar leaves out qualitygate.wait , so it reports rather than blocks.

None of the 829 tests crosses into ms-bpm , and none of them runs a BPMN gateway, a DMN table or a product flag. The volume moved in Bravo’s favour, but the question this deck asks — *does this change hit NDF2W?* — is untouched.

Where the business rules actually live, and why Mockito cannot reach them

In LORA we graded this complaint *overstated*, because LORA’s rules live in Go preconditions that `PreConditionIsMet(nil, data)` can run directly — 97 test files already do. In Bravo the complaint holds, because a large part of where decisions are made is **not in Java at all**.

WHERE A ROUTING DECISION LIVES	WHAT FORM IT TAKES	CAN A UNIT TEST RUN IT?
BPMN gateway conditions – 137 exclusive gateways in <code>ndf4w.bpmn</code> alone	JUEL expressions inside XML	No – only by deploying the process and running it. 4 test files do that
Which steps run for this application	<code>WorkflowProductConfig</code> , then <code>WorkflowMasterConfig</code> , then a per-application matrix in jsonb	No – they are rows in PostgreSQL, shipped as Flyway data migrations
The skip check itself	<code>BaseActivity.execute()</code> reads the matrix, then either runs or does nothing	Yes in principle – but <i>what is in the matrix</i> is still data
Which BPMN process starts for a product	A YAML map from <code>productId</code> to key at <code>application.yaml:1364</code> , with a Java override for R0	Partly – and the map contains four keys that are not deployed processes
Approval chain rules	<code>BaseUnderwritingApprovalServiceImpl</code> , 4,604 lines, behind feature flags	Yes – genuinely Java, and genuinely testable
Permanent error versus temporary one, and the decision to degrade	50 <code>customErrorHandle</code> implementations, called on the last retry	Yes for the handler itself. No for “does the engine ever actually reach the last attempt”

Bravo’s Java is testable, and largely tested. Bravo’s **decisions** live in XML, YAML and database rows — and no test in the build runs any of those.

WHY THAT IS EXPENSIVE

The `customErrorHandle` framework contains at least one **credit-policy** decision: after three failed attempts it skips anti-fraud, to keep the pipeline moving. Whether that path is ever reached depends on `failedJobRetryTimeCycle`, which is a string attribute on a BPMN element. There are **186 of them, with 30 different values, including 5 PT4M values that have no repeat count and so do not do what their author intended**. No test checks that any step’s retry policy is what the modeller meant — and the one that skips anti-fraud is the one where being wrong is a credit-risk event rather than an outage.

The product is identified in five different places

MECHANISM	WHERE
Numbers written into Java	<code>Application.isNDF4W() { return productId == 1L; }</code> – there are 18 of these
A YAML map of process keys	<code>application.yaml:1364</code> , plus an RO override in <code>ApplicationServiceImpl.java:752-762</code>
A six-column database selector	<code>WorkflowProductConfig</code> , then <code>WorkflowMasterConfig</code> , then a per-application matrix in <code>jsonb</code>
Java factories	31 of them
Gateway expressions written as strings	inside all 53 BPMN files

A change to any one of these can alter the behaviour of one product, several, or all of them. Adding DF2W Sharia (product 15) touched **all five**, behind per-product feature flags.

NOTHING CONNECTS A CHANGE TO THE PRODUCTS IT AFFECTS

There is nothing in the build – no test name, no tag, no line in a coverage report – that says which products a given change reaches. **A reviewer’s only tool is knowing all five mechanisms by heart.**

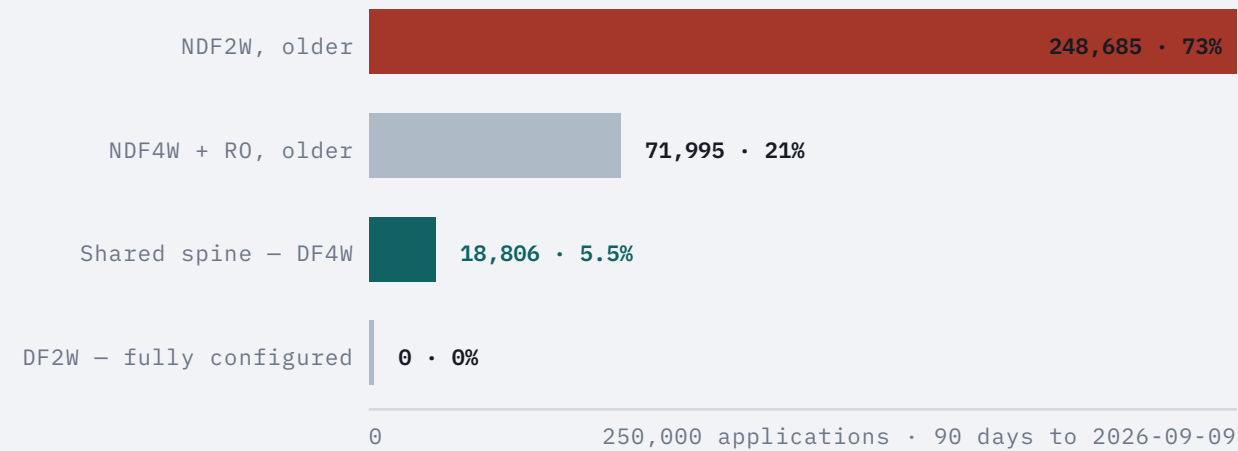
LORA’S ANSWER IS STRUCTURAL, NOT TESTED

Product families have separate document schemas, separate workers and separate task queues, so a change to `.../ndf2w` simply cannot reach NDF4W. LORA still cannot tell you *which* product a shared-planner change affects – that is its own True verdict – but it has fewer ways to get it wrong.

The bridge from the old workflows into the new ones

ndf2w.bpmn calls into the unified underwriting sub-process. And no callActivity anywhere in the codebase sets camunda:calledElementBinding , so every child resolves to the newest deployed version at the moment it is called.

NOW PUT THOSE TWO FACTS NEXT TO THE PRODUCTION VOLUME SPLIT



A change to unified-underwriting-*.bpmn — the “new” generation, which carries 5.5% of volume — is picked up by NDF2W loans that are already running, at their next call.

That is 73% of BFI’s origination volume, reached through a file whose name says it belongs to the other generation. There is no migration plan, no version pinned, and no test.

NOTHING COVERS THIS PATH

The four Camunda-running test files do not cover it, and neither does anything else. This is the most consequential untested path in either platform, and it is invisible when you review the change.

The models get checked in production, 38,198 times a week

The LORA complaint was that validation is “only JSON Schema, applied after the worker is already serving traffic”. Bravo has no schema registry at all, so the equivalent question is: *when does anything check a process model?* **When the pod starts, in production.** Spring Boot deploys every BPMN and DMN file it finds — there is no `deployment-resource-pattern` to narrow it — and the engine reads and versions each changed definition as it boots. **That read is the only check there is.**

A TYPICAL LINE, CAPTURED LIVE

```
status: warn
version: v2.93.53
message: ENGINE-09004 Warnings during parsing:
  * Exclusive Gateway 'Gateway_X_KYC_Verified' has
    outgoing sequence flow 'Flow_Condition_KYC_Verified'
    without condition which is not the default flow.
    We assume it to be the default flow, but it is bad
    modeling practice, better set the default flow in
    your gateway.
  | resource unified-kyc-check.bpmn
@activityName: KYC Check
@processDefinitionId: 544e3ea0-ab85-11f1-ba59-16deca89b476
```

Read that carefully. The engine is telling the team, *in production*, that a gateway on the **KYC path** has an outgoing route with no condition on it, and that Camunda is *guessing* this is the default. A routing decision is being settled by the engine’s assumption rather than by the model. The warning even names the file. It has been repeating across at least two deployed versions.

DEPLOYED VERSION	ENGINE-09004	WARNINGS
v2.92.45		24,513
v2.93.53		13,685
Total, 7 days		38,198
Measured on <code>env:prod service:prod-ms-bpm</code> , over the 7 days to 2026-09-10.		

1 • CHECKED AFTER DEPLOYMENT, NOT BEFORE

Every one of these warnings is available at **build time**. `camunda-bpm-assert` plus a parse test would surface them in seconds. Not one of them fails a build today.

2 • THE COUNT REFLECTS HOW OFTEN PODS RESTART

...and that is informative too. 38,198 in a week across two versions means **Pods are restarting often** and re-reading all 53 files each time. The number of *distinct* warnings is small. The volume tells you about pod churn.

3 • NOTHING READS THEM

No monitor matches `ENGINE-09004`. The warnings land in the same stream as the 525,181 error lines a week, and nobody sees them.

The honest comparison. LORA’s `docFieldCheck` crashes the *worker at startup* if a schema path is missing. That is late, but at least it is fatal and it happens on every deployment. Bravo’s equivalent check writes a warning, assumes a default, and serves traffic.

“Not a 5xx” means healthy

BRAVO DOES NOT HAVE LORA’S FAILURE MODE

About half of LORA’s loans never reach a finished status, while producing no errors and no failed attempts — every signal reads healthy. Bravo’s BPMN checkpoints **fail quickly, raise an incident and park the loan**, so a stuck Bravo loan becomes a visible ticket rather than an invisible running workflow. That is a real advantage, and it is why Bravo’s stuck-application rate is *measurable* at about 0.39%, while LORA’s silent half went unnoticed for months.

BRAVO’S VERSION OF THE SAME MISTAKE SITS ONE LAYER UP

```
(hits{service:prod-ms-bpm}
- hits{service:prod-ms-bpm, http.status_code:5*})
/ hits{service:prod-ms-bpm} < 99
```

Every health signal Bravo owns is defined as **the absence of a 5xx**.

Bravo’s release gate amounts to it deployed, and it is not returning 500.

That is the same kind of statement as **the workflow is still Running**, and it is wrong for the same reason: it defines health as the absence of the one failure mode the system does not have.

WHAT IS HAPPENING	WHAT THE HEALTH MODEL SAYS
748,686 HTTP 404s a week on the Surveyor Platform, 266,767 of them from one endpoint hitting 54,350 of 79,090 sessions	healthy
8,533 401 Unauthorized from the IAM permission endpoint, about 1,200 a day, 1.07 errors per trace – a wide authentication defect	healthy
20,802 outbound errors a week, none of them 5xx	healthy
About 4,750 step-level engine failures a day, with One Obligor Check at 5,583 in 48 hours	healthy – nothing reads @activityName
Eight monitors permanently in Alert, and six that cannot fire	–

The journey suite Bravo built, and stopped running

A CORRECTION TO THIS ANALYSIS

The first version of this deck concluded that *no* Bravo journey suite existed. It had checked three places — Datadog Synthetics, Datadog CI Visibility, and `bravo-bpm-service`. **It did not check sibling repositories, and there is one.** The corrected finding is *worse* for Bravo than the original, not better.

SQUADS/<SQUAD>/BRAVO-E2E-TEST	
Built with	Cypress and <code>@badeball/cypress-cucumber-preprocessor</code>
Feature files	595
Gherkin steps	28,268
History	600 commits by 46 people — 334 in 2022, 266 in 2023
Last commit	2023-11-21
CI workflows	4 — <code>OPERATION_PLATFORM</code> , <code>LOS_E2E</code> , <code>NDF4W_E2E</code> , <code>LMS_E2E</code>
Scenario tags	<code>@BLOS-T3424</code> , <code>@B4WH-T808</code> — the same Zephyr scheme as LORA's <code>@BL-T<n></code>

FEATURE FILES BY AREA	FILES
<code>surveyor-platform</code> — the console behind 28.4% of Bravo's tickets and the 404 flood	349
<code>agency</code>	114
<code>operation-platform</code>	55
<code>repeat-order</code>	39
<code>lms</code>	22
<code>ndf2w</code> — 73% of production volume	11
<code>ndf4w</code> · <code>unsecured</code>	3 · 2

The finding is not that nobody built a journey suite. Somebody built 595 of them, pointed 349 at Bravo's worst-performing console, wired up four CI workflows — and then **three things happened**.

A run that cannot fail, a suite that checks nothing, frozen for three years

ONE · ONLY ONE WORKFLOW IS SCHEDULED, AND IT CANNOT FAIL

```
on:
  schedule:
    - cron: "00 23 * * MON"    # weekly, not nightly
# the other three workflows only run when someone clicks
- run: |
  npx cypress run --browser electron --record \
    --spec ../BLOS-T3424.feature || true
# ..nine more specs, every one ending in  || true
```

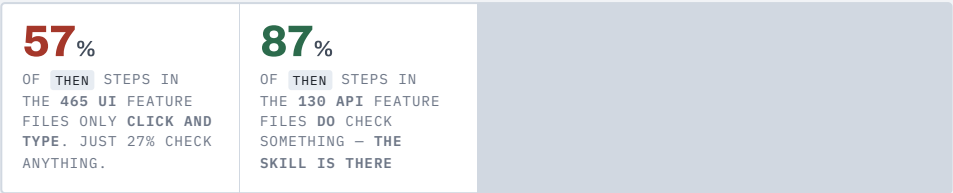
`|| true` THROWS AWAY THE EXIT CODE

The workflow **reports success whether the loan journey worked or not**, across 10 of the 595 files. This is the same defect as §5, one layer further out: Bravo’s release gate treats “not a 5xx” as healthy, and Bravo’s CI gate treats “the runner finished” as healthy.

A missing test cannot mislead anyone. A permanently green one can.

The original finding was an **absence**, and absences are easy to excuse — nobody got round to it. The corrected finding is **abandonment**: 46 people built this at scale, and it decayed while the console it covers became the largest single source of production tickets.

TWO · THE *UI HALF* CHECKS NOTHING — THE API HALF DOES



Re-measured 2026-09-10. Where the check was obvious — a status code, a response body — QA wrote it: Then the response status should be 200 . It broke down in the UI half, where nothing obvious presented itself, so Then became a word meaning “next”: Then Input username with text A Then that types into a field detects a crash and nothing else. And 463 of 595 files (78%) are named after whether the test is a positive or negative case, with only 101 distinct **Feature: names** — so the file list is not a table of contents of the loan business.

THREE · FROZEN, AND THE SCHEDULE HAS ALMOST CERTAINLY STOPPED

THIS IS AN INFERENCE, NOT A FACT

The last commit was 2023-11-21. GitHub turns off scheduled workflows after 60 days of repository inactivity, so the weekly job is very likely not running — and **zero CI pipeline events in 30 days** fits that. **We cannot confirm it from a local checkout.** Somebody has to open the Actions tab. *Five minutes, and it is the first item in the plan.*

AND THE 595 FILES ARE AN ASSET, NOT JUST A REPROACH

349 of them describe the surveyor journeys §7.3 wants regression cases for. They are Gherkin, so QA and product can read them, and they already carry Zephyr tags. **The L8 production test needs sorting and re-pointing, not writing from scratch — plus an exit code that means something.**

WHAT STILL STANDS

Bravo’s volume is its own smoke test: at 118,253 applications in August, a total break shows up in minutes. That works for outages. It does not work for the failure Bravo actually has — a slow, product-specific, 4xx-shaped decline, growing 82% over eight months while volume falls. **The continuous evidence is still the ticket queue.**

If the 349 surveyor files are being re-pointed, fix the writing style first

Re-pointing a file that names buttons, hard-codes waits and ends every `Then` with a click gives you a re-pointed file with the same defect. **Eight rules, each answering a measured number — and the first four can be checked automatically in about 50 lines of code over a parsed Gherkin file.**

RULE		VIOLATIONS
1. A <code>Then</code> may not contain an action verb. <code>Then</code> checks; <code>When</code> acts		7,418
2. A scenario about a business rule may not name a UI element — no button, field, dropdown or page		7,042
3. No <code>wait</code> in a step. Time appears only as an SLA or an expiry rule		618
4. Use <code>Scenario Outline</code> only when <code>Examples</code> has at least 2 rows that change the outcome		1,007 of 1,068
5. <code>Feature:</code> names a business capability, never whether the test is positive or negative		463 of 595
6. One scenario, one rule. If the name needs “and”, split it		97 run over 100 steps
7. Step text in English; Indonesian only inside quoted values		1,349
8. A capped, reviewed list of step phrasings — the <code>When</code> list <i>closed</i>		2,438 phrasings

WHY A STYLE CHECK COMES BEFORE THE SORTING

The suite decayed this way once already. 1,229 of its 2,438 step phrasings are used exactly once, behind 1,691 step definitions across 206 files — so the glue code is one-to-one with the prose, and every re-pointed file drags its own bespoke steps along with it. There is even a branch on the remote called `B4WH-5851-qa-split-object-and-action` : **QA diagnosed this themselves, and the fix never landed.**

TWO TIERS, AND THE RATIO BETWEEN THEM IS THE REAL FIX

Many small scenarios that check a state, a status or a rejection reason. These belong at L3 to L5. And single digits of full UI journeys per product, at L7 to L8. Collapsing the two is how a scenario ends up 319 steps long, and a scenario that long cannot fail for one clear reason. **Bravo’s own API features are the first tier done correctly — the model to copy is in the same repository.**

KEEP ONE THING EXACTLY AS IT IS

Every scenario carries a Zephyr key — `@BLOS-T3424` , `@B4WH-T808` — under a `@ProjectKey` feature tag. It is the same scheme LORA enforces as `@BL-T<n>` . **It links a requirement to a test run, and it is the one practice in this suite that needs no correction.**

Sections 1 to 6 are the diagnosis. This is the structure.

The diagnosis on its own has a weakness: **nine recommendations with no structure to hang them on become nine tickets, each argued separately.** LORA’s *lora-super-test* does have a structure – five layers prefixed with `_` (`_whitebox` , `_canary` , `_contract` , `_widgets` , `_utility`), a four-row cost budget, and a Zephyr tag on every case. It also says *why*: running everything end to end against SIT “*tested their uptime, not our code*”. **Bravo has no equivalent structure at all.** What follows is built from Bravo’s own decisions rather than copied from LORA.

DESIGN RULE 1 – FROM LORA, AND STILL TRUE HERE

Test only what you control, and own the answers at the boundary. LORA’s boundary is a single gateway envelope. Bravo’s is different, and it is better in one way and worse in another.

DESIGN RULE 2 – THE WHOLE POINT FOR BRAVO

A change should fail in the cheapest layer that can see it. LORA states this as a rule of thumb. For Bravo it carries real weight, because Bravo’s *expensive* layer – a Camunda process test – is the only layer that exists for the workflow today, and **it exists four times.**

WHERE BRAVO’S TEST BOUNDARY ACTUALLY SITS

	LORA	BRAVO
Shape of a call to an upstream	One envelope, <code>POST /proxy/inner/request-v1</code> , naming its schema	113 typed <code>@FeignClient</code> interfaces , one per upstream
Where you intercept it	One mock, matching on <code>api_data_schema</code> , answers for about 300 upstreams	113 sets of stubs – more of them , but each is a typed Java interface
What catches a stub drifting out of date	<code>check:stubs</code> , run against the live LSS schema registry	The compiler. Bravo has no schema registry – but a stub built from the Feign type <i>cannot</i> drift without breaking the build
Can you make an upstream fail on demand?	Yes, per run – a spec changes one answer in two lines	No. <code>bravo-mock-service</code> (Mockoon, at <code>*.mock.bravo.bfi.co.id</code>) is fixed and shared – the same weakness LORA names when explaining why it built its own

The finding is in the last two rows, and they point in **opposite** directions.

Bravo’s boundary needs no schema registry to stay honest, because 113 typed interfaces are checked by `javac`, where LORA’s 300 JSON envelopes need a purpose-built script. **That is a real advantage, and Bravo got it for free.** But Bravo has **no way to make an upstream fail on demand** – and the failure suite is the list of what that costs.

The nine layers, ordered by cost

Each row says what **only** that layer can catch. A layer that catches nothing a cheaper one already sees does not belong here.

LAYER	WHAT IT COVERS	WHAT ONLY IT CAN CATCH	COST	TODAY
L0 · model lint	All 53 BPMN and 3 DMN files read at build time, with no Spring context	ENGINE-09004 (38,198 a week in production); the 5 bare PT4M retry cycles among 186 declarations; the 4 YAML process keys that are not deployed processes; the 56 callActivity elements with no calledElementBinding; orphans among 70 error definitions and 190 escalations	seconds	none
L1 · config-table checks	The six-column selector, the productId -to-key YAML map, the per-application jsonb matrix, and 1,350 Flyway migrations	That every productId resolves to a deployed process; that every step named in a jsonb matrix exists among the 274 distinct bean names; that no WorkflowMasterConfig row points at nothing	seconds	none
L2 · Java unit tests	The 776 Mockito tests that already exist. BaseUnderwritingApprovalServiceImpl (4,604 lines) and the 50 customErrorHandle bodies	Ordinary Java defects. This layer is fine – it is the one Bravo already has	ms	776 files
L3 · gateway expressions	Every conditionExpression string, evaluated through ExpressionManager against a set of variables – with no process started	Which branch a given set of variables takes, across 137 exclusive gateways in ndf4w.bpmn alone, and about 1.3 conditions per service task. This is the layer that turns Bravo’s routing back from data into code	ms	none
L4 · step contracts	Each of the 276 JavaDelegate classes, against stubbed Feign clients	What a step writes back into the process; which customErrorHandle fires on the last attempt – including the anti-fraud BYPASS credit decision	ms	partly, inside L2
L5 · process fragments	One sub-process deployed and driven with camunda-bpm-assert – unified underwriting on its own, unified KYC on its own	Retries running out, an incident being raised, escalation across one boundary, and the forced-termination path	~1 s	4 files
L6 · product journeys	One process test per generation and product, from start to go-live, with upstreams stubbed	“Does this change reach NDF2W?” – the question §3 says nothing answers. And the bridge from ndf2w.bpmn into unified underwriting	~10 s	none
L7 · console contracts	A snapshot of the response shape of the 9 or more FormTab endpoints each console pane depends on, and one live test per shared component	A console change that 404s a pane; the shape change behind Cannot read properties of null (6,445 a week)	~2 s	none
L8 · production check	A Synthetics multi-step API test that creates a real application, plus the errors-per-view SLO for each console	That origination works right now, on real infrastructure, with real upstreams	minutes	8 DNS/SSL

Six of the nine layers do not exist — and the two that would have caught the most both run in milliseconds.

L0 and L3 are the two Bravo has never built, and between them they cover every finding in §2: the XML, the YAML and the config rows that Mockito cannot reach are all reachable without starting the engine.

When something breaks, which layer should have caught it

SYMPTOM	CHEAPEST LAYER THAT CAN SEE IT	EXISTS?
A gateway has no default route	L0	no
A retry cycle never repeats	L0	no
A new product has no deployed process	L1	no
A jsonb matrix names a step that no longer exists	L1	no
An approval-ladder rule is wrong	L2	yes
A change reroutes one product and not another	L3	no
A step writes the wrong variable	L4	partly
An upstream failure does not raise an incident	L5	4 files
A unified edit changes the path of a running NDF2W loan	L6	no
A console pane 404s	L7	no
Origination is broken in production	L8	DNS only

8 of 11

SYMPTOMS HAVE NO LAYER AT ALL – AND THE TWO IN BOLD CARRY THE LARGEST PRODUCTION EXPOSURE IN THIS DOCUMENT

THIS IMPLIES A DIFFERENT ORDER FROM THE RECOMMENDATION LIST – AND A BETTER ONE

L0 AND L1 SHOULD COME FIRST

Recommendation 2 is written as “fail the build on ENGINE-09004”. But the test setup it needs is L0, and once L0 exists it **also** delivers T1, T3, T12 and the detection half of recommendation 1, within the same few days. **L0 and L1 are the cheapest work in this document, and they close the largest number of findings.** They belong ahead of the item currently marked highest-value.

THE RULE OF THUMB, RESTATED FOR BRAVO

A form change fails in L7 in two seconds. A routing change fails in L3 in milliseconds. Only a genuine break in the flow should ever cost an L6 journey – which is why there are exactly **five** of those.

The regression suite, T1 to T12

A regression test exists *because something happened*. Every case below is tied to evidence in this pack, and the layer column says where it belongs. **Not one of them needs a browser.**

#	CHECK THAT...	BECAUSE	LAYER
T1	Gateway_X_KYC_Verified has an explicit default route	The engine is guessing it in production, on the KYC path, across two deployed versions	L0
T2	An NDF2W loan enters a pinned version of underwriting	ndf2w.bpmn bridges into unified underwriting; 56 call activities are unpinned; 248,685 applications in 90 days	L6
T3	Every failedJobRetryTimeCycle is a valid Rn/PTn	186 declarations, 30 different values, 5 bare PT4M that never repeat	L0
T4	Anti-fraud skips to BYPASS only on the last attempt	A credit-policy decision reached through an exception handler, verified by nothing	L4 + L5
T5	feature-configuration returns the status it is documented to return	266,767 404s a week across 69% of surveyor sessions , on the busiest handler in the service	L7
T6	Every key in the productId-to-process map is a deployed process	The map contains four that are not	L1
T7	The approval ladder still resolves after a role is renamed	BLCS-4683 renamed NMH to GMB; BLCS-4811 is reversing it	L1 + L2
T8	The IAM token is refreshed before calling /permission/assigned	8,533 401s a week, at 1.07 per trace – a wide authentication defect, not a retry loop	L4
T9	Adding a product touches all five identification mechanisms consistently	Product 15 (DF2W Sharia) touched all five, behind flags, with no tests	L1 + L3
T10	Position capture succeeds, or fails in a way the user can see	About 20,700 location failures a week on a console used by field staff	L7
T11	The FormTab v2 set of panes matches its snapshot	9 or more endpoints; four of them are the top sources of the 404 flood	L7
T12	Every monitor's service: tag matches a registered service	Four Sharia monitors filter on prod-sharia-bpm against prod-sharia-bpm-sharia, and cannot fire	L0
Eight of the twelve land in L0, L1 or L7 – layers that run in seconds. Only T2 needs the expensive layer.			

The failure suite, N0 to N12

#	MAKE THIS HAPPEN	THEN CHECK THAT...	LAYER
N0	An upstream returns 5xx	it retries according to <code>failedJobRetryTimeCycle</code> , then raises an incident and parks the loan	L5
N1	An upstream returns 401	the token is refreshed and the call retried, rather than counted as a permanent failure	L4
N2	An upstream returns 404	it is treated as “not there” rather than as an error – the live repeat-order case, 5,070 a week	L4
N3	An upstream returns 400	a broken contract is permanent and named, not retried 186 times	L4
N4	An upstream never answers	the 300-second global <code>readTimeout</code> is not the timeout that actually applies to a job-executor thread	L4
N5	Retries run out	the right one of the 50 <code>customErrorHandle</code> implementations fires	L4 + L5
N6	Anti-fraud fails three times	it falls back to <code>BYPASS</code> – and that is recorded as a credit decision , not as a log line	L5
N7	A well-formed payload has a field of the wrong type	it is rejected at the client, and never written onto the application record	L4
N8	The jsonb matrix switches off a mandatory step	the process refuses to start, rather than skipping a required check	L1 + L5
N9	A child sub-process throws	the escalation reaches the parent, through one of the 190 escalation paths	L5
N10	A loan is forcibly terminated part-way through	its status and <code>application_status_log</code> agree, and no job is left orphaned	L5
N11	The same application is started twice	the second start changes nothing – one process instance, and an unambiguous <code>RetryLog</code>	L4
N12	Two approvers decide at the same moment	one wins, and the ladder does not skip a tier	L2

Bravo cannot make an upstream fail at all.

LORA can make any upstream fail, per run, by changing two lines. `bravo-mock-service` is fixed and shared, so “have anti-fraud reject this one” is not something a Bravo test can do. That is why 50 `customErrorHandle` implementations, 70 error definitions and 190 escalation paths are never exercised.

AND BRAVO’S VERSION IS THE CHEAPER ONE TO BUILD

N0 to N12 need one piece of infrastructure, and it is the same piece LORA built: a **per-run stub layer in front of the 113 Feign clients**, where a test changes one answer and everything else comes from a shared baseline. Bravo’s stubs are generated from *typed interfaces*, so there is **no schema registry to keep in step** and **no `check:stubs` script to write**.

```
// the shape a Bravo failure case should have
stubs.baseline(NDF2W_SCORING_1);
stubs.antiFraud().failsWith(503).times(3);
// then check the fallback, not the log line:
assertThat(application)
    .hasCreditDecision(BYPASS)
    .recordedBy("customErrorHandle");
```

400 cases must not mean 400 process runs

LORA’s arithmetic applies here unchanged: **400 cases at a 2-minute walk each is 13 hours**, and a suite nobody can run inside a sprint is a suite nobody runs. Here are the same 400 cases spread down the ladder:

LAYERS	CASES	EACH	RUN WHEN	TOTAL TIME
L0 + L1 – lint and config	~60	~20 ms	every build	~2 s
L2 + L3 + L4 – Java, expressions, steps	~1,100	~5 ms	every build	~15 s
L5 – process fragments	~60	~1 s	every build	~1 min
L6 – product journeys	5	~10 s	every merge	~1 min
L7 – console contracts	~40	~2 s	every merge	~1.5 min
L8 – production check	1	~3 min	hourly	–
Total per merge				≈ 4 min

Five L6 journeys covers the whole product range: NDF2W, NDF4W, RO, unified DF4W and DF2W.

Five tests would cover the normal path across 100% of production volume, against the zero that cover it today. They are the expensive layer, deliberately kept to five, because L0 to L4 catch everything that does not need the engine.

§7.6 · THREE RULES THAT STOP IT DECAYING

1 · NO HAND-WRITTEN LISTS OF WHAT TO EXPECT

LORA works out its 24-stage plan from real traces, because hand-written lists “*were proven wrong twice*”. Bravo’s equivalent is the set of steps each product runs: **derive it from act_hi_actinst, never type it out**. This depends on the job-executor traces in bravo-observability.md rec 8 – **without them, a derived Bravo expectation has nothing to derive from**. That is the one dependency this document has on another.

2 · NO SELECTORS OR FIELD PATHS INSIDE A TEST

Bravo’s version: **no productId == 1L written into a test**. A test asks the config layer which product it is, so that a test cannot become the sixth way the product is identified.

3 · NO STUB COPIED INTO A CASE

Name a baseline bundle and state only what differs. That is what makes N0 to N12 two lines each, instead of a fixture per case.

AND ONE RULE THAT IS BRAVO’S ALONE

A snapshot is never recorded automatically. LORA states the reason precisely: if the first passing run records the baseline, “*that day’s form – bugs included – would silently become the truth*”. For Bravo the trap is bigger, because **L0 would otherwise record 38,198 parse warnings a week as the approved state of the models**.

Fifteen actions 1 – 7

1 Pin <code>calledElementBinding</code> , or write the one test that proves the bridge works · S-M, HIGHEST VALUE There are 56 call activities, none pinned to a version, and <code>ndf2w.bpmn</code> calls into unified underwriting. Either pin the version, so a unified edit cannot reach 73% of volume part-way through a loan, or add a process test that starts an NDF2W loan and checks which underwriting version it enters. Today neither exists.	2 Fail the build on <code>ENGINE-09004</code> · S The warnings already exist. They are simply produced in the wrong place. A parse test over all 53 files using <code>camunda-bpm-assert</code> , which is already in the pom, turns 38,198 production warnings a week into a red build. Start with the KYC gateway the engine is currently guessing about.
3 Make the release health gate count 4xx · S “Not a 5xx” is not health. At a minimum, track 404 and 401 rates per endpoint on the four surveyor endpoints and the IAM permission call.	4 Build one end-to-end process test per generation · M Two tests: one NDF2W loan and one unified DF4W loan, each walked from start to go-live with upstreams stubbed, checking the final status and which steps ran. Two tests would cover the whole normal path of both generations, which nothing covers today.
5 Test the retry and fallback policies · M Check that <code>failedJobRetryTimeCycle</code> is a valid <code>Rn/PTn</code> on every service task — the 5 bare <code>PT4M</code> cycles are a defect waiting to happen — and drive a step to its last attempt to see which <code>customErrorHandle</code> fires. The anti-fraud <code>BYPASS</code> is a credit-policy decision reached through an exception handler, and nothing verifies when it happens.	6 Show in the change itself which products it reaches · M Tag every test with the products it covers, and for each pull request print the set of <code>productId</code> s reachable from the changed BPMN files, config rows and factories. A reviewer should not have to know five identification mechanisms to know whether a change reaches NDF2W.
7 Turn on a coverage gate, and fix the <code>makefile</code> · S Use JaCoCo <code>check</code> with a floor at the current level, then raise it step by step. And fix <code>-Dspring-boot.run.profiles</code> , which surefire ignores.	

Fifteen actions 8 – 15, and where each one lands on the ladder

8 Add one Synthetics multi-step API test for the origination journey · S-M Not a replacement for a process test — a running check against production. Today the only continuous evidence that Bravo works is the support-ticket queue.	9 Send CI events to Datadog · S Neither platform has a single pipeline event, so nobody can answer “is the build getting slower, flakier, or redder?” It is a configuration change, and it helps both teams.
10 Build the per-run stub layer in front of the 113 Feign clients · M — UNBLOCKS N0-N12 Today no Bravo test can make an upstream fail, so 50 customErrorHandler implementations, 70 error definitions and 190 escalation paths are never exercised . bravo-mock-service (Mockoon) does not solve this — it is fixed and shared, which is exactly why LORA built its own. Bravo's version is cheaper: stubs are generated from typed Feign interfaces, so there is no schema registry to keep in step.	11 Derive what you expect, never type it · S — A POLICY, NOT A TASK LORA works out its plan from real traces, because hand-written lists were proven wrong twice. Bravo's equivalent is the set of steps each product runs, derived from act_hi_actinst — which needs the job-executor traces in bravo-observability.md rec 8 first. Until then an L6 journey's expected step list has no trustworthy source , and that is this document's one dependency on another.
12 Delete every true from OPERATION_PLATFORM.yml , then decide what to do with the suite · S TO FIX, M TO SORT OUT — DO THE FIX TODAY The one scheduled Bravo end-to-end workflow cannot report a failure. That is a two-character deletion per line, and it turns a decorative run into a real one. Then open the Actions tab and find out whether the weekly job has run at all since 2023 — five minutes, and this deck cannot answer it from a checkout. After that the 595 files are a sorting job, not a rewrite: 349 cover the console generating 28.4% of tickets, and they already carry Zephyr tags.	13 Ship the four automatic Gherkin rules as CI before the sorting starts · S No action verb in a Then ; no UI element in a rule scenario; no wait ; no Scenario Outline with fewer than two Examples rows. That is about 50 lines of code over a parsed Gherkin file, and it will fail most of the existing suite on contact — which is the point. Without it the sorting is a one-off, and this suite has already decayed this way once.
14 Rotate the credentials committed in cypress.config.js · S — A SECURITY TASK, NOT A TESTING ONE Found while measuring the suite: a Jira user API token, a Zephyr Scale API key, an LMS username and password in plain text , an agreement apiSecret and a lead token — in a repository with 46 contributors and two squad checkouts. The JWTs expire in 2024; the plain-text credentials never expire. Assume the whole history is exposed: the values are in 600 commits, not just the current files. Escalated 2026-09-10 — a possible root cause, not just hygiene. The production engine holds 61 injected process definitions designed to run code, one confirmed to have run , and the route in requires a credential . This repository is the most plausible source anyone has identified , and the intrusion window (2026-05-23 to 06-30) sits inside the exposure period. Not established — but this is now the second action in an active security incident .	15 Enforce the coverage the three consoles already measure · S All three compute coverage on every pull request and send it to SonarQube, and all three set the failing threshold to 0% or 1% , so it cannot fail. Read the current lcov figure, set the threshold just below it, and let jest-coverage-thresholds-bumper — already a dependency, never called — raise it over time. Add -Dsonar.qualitygate.wait=true in the same change. The cheapest quality win in the estate: tests, runner, coverage report and blocking job all exist. One number in three files is the difference between measuring and enforcing.

Which layer each action buys

The order the ladder implies is **different from the list, and better**: L0 and L1 are the cheapest work here, and between them they carry recommendations 1, 2, 5 and 6.

REC	LAYER	NOTE
2	L0	Also delivers T1 , T3 and T12 in the same test setup
7	L2	Puts a gate on the layer that is already healthy
6	L1 + L3	Falls out of L1 and L3 almost for free once they exist – the set of reachable <code>productId</code> s is the L3 evaluation
5	L0 then L4/L5	Part 1 is L0 and costs 4 days; part 2 needs L5 <i>and</i> recommendation 10
1	L0 → L6	L0 finds all 56 in seconds; only L6 proves the fix
4	L6	Five journeys, not two – the full product range
3	L7	Together with T5 , T10 and T11
8	L8	–
9	–	Infrastructure for all of it: today no layer’s run time can be measured
10	enables L4, L5	The whole of N0 to N12
11	policy for L6	Blocked on observability recommendation 8

30, 60, 90 days

Eleven recommendations, phased against **about 25 person-days a month** of Squad S&U time — the allocation set by bravo-people.md recommendation 4. **The phases are the ladder, built cheapest first:** L0 and L1 in month 1, L3 and L4 in month 2, L5 and L6 in month 3. That order is not a preference: *L6 is the only layer that can prove the `calledElementBinding` fix, and L0 the only one that can find all 56 call sites in seconds.*

THE RULE FOR ORDERING THIS WORK

Investigate before touching anything that reaches loans already running. Recommendation 1 is the highest-value item here and the most dangerous to rush: unpinned `callActivity` versions affect **248,685 NDF2W applications every 90 days**. So it is read-only in month 1, a decision in month 2, and a change in month 3. Nothing else in this plan alters the behaviour of a running loan at all.

SHARED WITH OTHER DOCUMENTS

Recommendation 1 is also rec 6 in **bravo-delivery.md**, and recommendation 3 is also rec 2 in **bravo-observability.md**. Both are phased the same way everywhere, so do them once.

Days 0–30

LOOK, AND START REPORTING

- 1
- Investigate `calledElementBinding` — read-only. List all 56 `callActivity` elements, map which children an NDF2W loan reaches, and size the exposure. *S&U · 5 days*. A written note. **No code change this phase.**
- 9
- Send CI pipeline events to Datadog. *Platform · 2 days*. Zero pipeline events becomes a build-health number, for both platforms.
- L0
- Build L0 — model lint. All 53 BPMN and 3 DMN files read at build time with `camunda-bpm-assert`, already in the pom. *S&U · 4 days*. Delivers rec 2, plus **T1**, **T3**, **T12** and the *detection* half of rec 1. **38,198 production parse warnings a week become a red build that runs in seconds.**
- L1
- Build L1 — config-table checks. *S&U · 3 days*. The **four YAML keys that are not deployed processes** fail the build. **T6**, **T7** and **T9** land here.

Days 31–60

PUT THE GATES IN

- 2
- Fail the build on `ENGINE-09004`, and fix the KYC gateway whose default route the engine is guessing. *S&U · 4 days*.
- 3
- Make the release health gate count **4xx**. *Platform and SRE · 3 days*. A release that 404s two-thirds of sessions fails the gate.
- 7
- Coverage gate, and the `makefile` fix. *S&U · 2 days*.
- 8
- A Synthetics multi-step API test for origination. *QA · 3 days*.
- 1
- Decide `calledElementBinding` — pin it, or decide deliberately that running loans move to the new child, with a migration plan. *S&U and EM · 2 days*.
- L3
- Build L3 — gateway expression evaluation. Every `conditionExpression` evaluated through `ExpressionManager`, with no process started. *S&U · 5 days*. **Bravo's routing stops being data and becomes code** — and this plus L1 is recommendation 6.
- 10
- Build the per-run stub layer in front of the 113 Feign clients. *S&U · 6 days*. An upstream can be made to fail on demand for the first time. **Unblocks all of N0 to N12.**

Days 61–90

PROVE THE JOURNEY, THEN MAKE THE CHANGE

- L6
- Build L5 and L6 — the five product journeys. NDF2W, NDF4W, RO, unified DF4W and DF2W, each from start to go-live on the phase-2 stub layer. Expectations **derived** from `act_hi_actinst`, never typed out. *S&U and QA · 12 days*. The journey question is answered **for 100% of production volume** instead of 0%. **T2** lands here.
- 1
- Carry out the binding decision — pin all 56 call activities, using the migration plan from phase 2. *S&U · 10 days*.
- 5
- Retry-policy checks, part 1. Check that every `failedJobRetryTimeCycle` is a valid `Rn/Ptn`. *S&U · 4 days*.

Deferred

WITH TRIGGERS

- N
- The failure suite, **N0 to N12**. Force each kind of upstream failure and check what the system falls back to. **N6** — the anti-fraud `BYPASS` — is a credit-policy decision reached through an exception handler. Needs *both* the phase-2 stub layer and the phase-3 L5 setup; about 2 days per case once both exist. *Trigger: once the L6 journeys land*.
- L7
- L7 — console contracts**. Snapshot the shapes of the 9 or more `FormTab` endpoints, plus one live test per shared component. Covers **T5**, **T10** and **T11**. The phase-2 4xx gate *detects* these; L7 *prevents* them. *Trigger: the console inventory in bravo-delivery.md rec 7*.
- 6
- Show which products a change reaches, in the change itself. 8 days, and much cheaper once job-executor tracing and the flag register exist. *Trigger: Q2 week 1*.

What changes when this is done

RECOMMENDATION	EXAMPLE WHEN DONE
<code>calledElementBinding</code> pinned	A change to unified underwriting cannot silently alter the path of 248,685 NDF2W loans that are already running. Migration becomes a decision instead of a side effect.
<code>ENGINE-09004</code> fails the build	The KYC gateway whose default route Camunda is currently guessing gets fixed in a pull request, rather than discovered in a log stream nobody reads.
4xx counted in the health gate	A release that starts returning 404 to two-thirds of surveyor sessions is caught at deploy, not by a rising ticket category.
Two end-to-end process tests	“Can a loan still get from submission to go-live?” has an automatic answer for both generations, on every build.
Retries and fallbacks tested	Nobody finds out from an incident that anti-fraud has been skipped after three failures, or that five retry cycles never repeat.
Products reached, shown in the change	A reviewer sees “this pull request reaches NDF2W, NDF4W and R0” on the pull request itself, and the five identification mechanisms stop being folklore.
CI events flowing	Build health becomes a number for both platforms, instead of an impression.
L0 and L1 exist	A gateway with no default route, a retry cycle that never repeats, a product with no deployed process, and a jsonb matrix naming a deleted step all fail the build in seconds – instead of in a production log stream nobody reads.
L3 exists	“Which products does this change reroute?” is answered by evaluating the gateway conditions, not by a reviewer who happens to know all five identification mechanisms.
The stub layer exists	A test can say “ <i>on this run, anti-fraud rejects</i> ”. 50 <code>customErrorHandle</code> implementations, 70 error definitions and 190 escalation paths stop being untested.
The ladder has a routing rule	A form change fails in L7 in two seconds, not in a 10-second journey three layers away from the cause – and the per-merge suite still finishes in about four minutes .

Today the build can be green for a change that reroutes 94% of production volume. Two tests and one attribute close that.